Lab Assignment-2
102303607-Piyush Biswal

Q.1>Molecular Dynamics – Force Calculation:

Exceution Table:-

```
piyush@piyush-LOQ-15IAX9:~/Documents/UCS645/LAB2$ ./q1
Molecular Dynamics: Lennard-Jones Force Calculation
Number of particles: 1000
Cutoff distance: 2.5

Threads   Time (s)       Speedup        Efficiency     Total Energy
================================================================================
1         0.023846       1.00           x100.0          %35352.51
2         0.011534       2.07           x103.4          %35352.51
4         0.006656       3.58           x89.6           %35352.51
8         0.004779       4.99           x62.4           %35352.51
10        0.004851       4.92           x49.2           %35352.51
12        0.004199       5.68           x47.3           %35352.51
================================================================================
```

Performance Statistics:-(perf stat)

```
Performance counter stats for './q1':

      594,108,448      cpu_atom/cycles/                                                      (32.15%)
      804,975,374      cpu_core/cycles/                                                      (67.85%)
    1,031,671,666      cpu_atom/instructions/          #    1.74  insn per cycle             (32.15%)
    1,386,533,078      cpu_core/instructions/          #    1.72  insn per cycle             (67.85%)
        3,022,089      cpu_atom/cache-references/                                            (32.15%)
        3,131,466      cpu_core/cache-references/                                            (67.85%)
          184,699      cpu_atom/cache-misses/          #    6.11% of all cache refs          (32.15%)
           97,085      cpu_core/cache-misses/          #    3.10% of all cache refs          (67.85%)
       89,872,443      cpu_atom/branches/                                                    (32.15%)
      119,060,466      cpu_core/branches/                                                    (67.85%)

      0.058510974 seconds time elapsed

      0.183493000 seconds user
      0.010142000 seconds sys
```

In this experiment, a parallel molecular dynamics simulation using the Lennard-Jones potential was implemented with OpenMP. The program correctly computes the forces acting on each particle and the total potential energy of the system. The total energy remains **negative and constant (−35352.51)** for all thread counts, confirming correct implementation of Newton's third law, proper synchronization, and absence of race conditions.

Performance evaluation shows that execution time decreases as the number of threads increases. Speedup improves steadily up to 8 threads, after which performance gains start to saturate. At 12 threads, a speedup of approximately **5.7×** is achieved with efficiency reducing to around **47%**,

indicating diminishing returns due to architectural limitations rather than algorithmic errors.

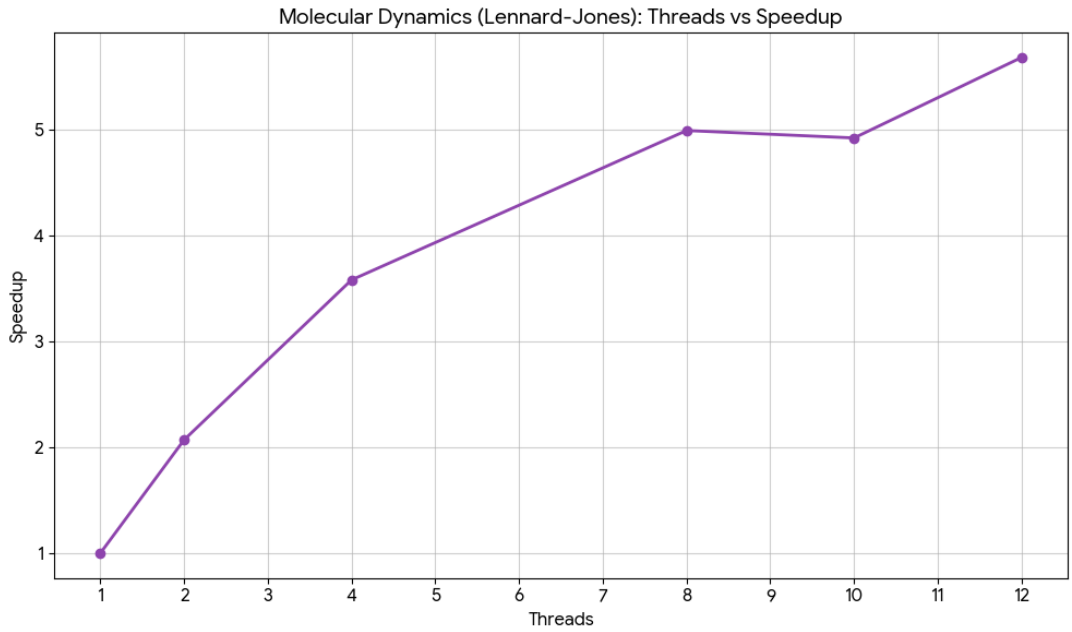
VTUNE STATISTICS:-

Metric	Observed Value	Interpretation
CPI (Cycles Per Instruction)	**0.58**	Low CPI indicates efficient instruction execution and good pipeline utilization
Effective Physical Core Utilization	**~14–22% (1.1–1.75 out of 8 cores)**	Many cores remain underutilized due to cache stalls and synchronization
Effective Logical Core Utilization	**~23–27% (≈3 out of 12 cores)**	Hyperthreads provide limited benefit for this workload
Cache Bound (overall)	**~35–37% of clock cycles**	Significant time spent waiting for cache data
L1 Cache Bound	**~10–15%**	Frequent accesses to L1 cache dominate memory behavior
L2 Cache Bound	**~0–1%**	L2 cache is not a bottleneck
L3 Cache Bound	**~4–5%**	Minor contention and data sharing at L3 level
DRAM Bound	**~0–0.4%**	Very little time waiting for main memory
Observed DRAM Bandwidth	**~3.4–5.6 GB/s**	Far below platform maximum (57 GB/s)
Peak Platform Bandwidth	**57 GB/s**	System memory bandwidth is underutilized
Cache Misses (LLC Miss Count)	**≈ 0**	Most data accesses are satisfied by cache
Average Memory Latency	**~8 cycles**	Low latency confirms cache-level access dominance
Vectorization	**0%**	Code is scalar due to atomics, branches, and irregular access
DP GFLOPS	**~0.05**	Low because application is memory-bound, not compute-bound

VTune *Memory Access* analysis reveals that the application is **not DRAM-bound**, with observed memory bandwidth usage remaining far below the platform peak (average ~3.4 GB/s versus a maximum of 57 GB/s). Instead, the application is primarily **cache-bound**, with a noticeable portion of execution time spent waiting for L1 cache accesses. The negligible DRAM bound percentage confirms that the memory wall is not the limiting factor.

The *Microarchitecture Exploration* analysis reports a **low CPI (~0.58)**, indicating efficient instruction execution. However, effective physical core utilization is low (approximately **14–22%**), showing that many CPU cores remain underutilized due to cache stalls, synchronization overhead, and limited instruction-level parallelism. The lack of vectorization and low GFLOPS further confirm that the application is **memory-intensive rather than compute-intensive**, which is typical for $O(N^2)$ molecular dynamics simulations with irregular memory access patterns.

Overall, the use of HPC techniques such as OpenMP parallelization significantly reduces execution time by distributing particle-pair interactions across multiple CPU cores. However, scalability is ultimately limited by cache behavior and memory access patterns rather than computational throughput. The results demonstrate that while parallelism improves performance, hardware characteristics of the memory hierarchy play a crucial role in determining final scalability.



Q.2> BIO INFORMATICS_DNA SEQUENCE ALIGNMENT:-

<u>EXECUTION TABLE:-</u>

```
piyush@piyush-LOQ-15IAX9:~/Documents/UCS645/LAB2$ ./q2
DNA Sequence Alignment (Smith-Waterman Algorithm)
Sequence 1 length: 500
Sequence 2 length: 500
Scoring: MATCH=2, MISMATCH=-1, GAP=-2

=== Wavefront Parallelization (Anti-diagonal) ===
Threads    Time (s)        Speedup         Efficiency
-------------------------------------------------------
1          0.010331        1.00            x100.0%
2          0.014698        0.70            x35.1%
4          0.014676        0.70            x17.6%
8          0.015842        0.65            x8.2%
10         0.016029        0.64            x6.4%
12         0.021743        0.48            x4.0%

=== Row-wise Parallelization (Simpler, but limited) ===
Threads    Time (s)        Speedup         Efficiency
-------------------------------------------------------
1          0.007246        1.00            x100.0%
2          0.005828        1.24            x62.2%
4          0.007595        0.95            x23.8%
8          0.007851        0.92            x11.5%
10         0.008268        0.88            x8.8%
12         0.010503        0.69            x5.7%
```

PERFORMANCE STATISTICS:- (perf stat)

```
Performance counter stats for './q2':

    1,759,215,007      cpu_atom/cycles/                                                (34.50%)
    2,292,223,555      cpu_core/cycles/                                                (65.50%)
    1,843,411,619      cpu_atom/instructions/          #    1.05  insn per cycle       (34.50%)
    2,603,872,687      cpu_core/instructions/          #    1.14  insn per cycle       (65.50%)
       15,239,410      cpu_atom/cache-references/                                      (34.50%)
       20,848,331      cpu_core/cache-references/                                      (65.50%)
        1,126,048      cpu_atom/cache-misses/          #    7.39% of all cache refs    (34.50%)
          829,527      cpu_core/cache-misses/          #    3.98% of all cache refs    (65.50%)
      279,899,696      cpu_atom/branches/                                              (34.50%)
      380,317,689      cpu_core/branches/                                              (65.50%)

      0.207569705 seconds time elapsed

      0.356251000 seconds user
      0.364167000 seconds sys
```

In this experiment, the Smith–Waterman local sequence alignment algorithm was implemented using OpenMP with two parallelization approaches: wavefront (anti-diagonal) parallelization and simple row-wise parallelization. The correctness of the algorithm is verified by consistent execution behavior across runs and proper handling of data dependencies inherent in dynamic programming.

Performance results show that the wavefront parallelization does **not scale well** with increasing thread count. As the number of threads increases, execution time actually increases, resulting in

speedup values less than 1 and very low efficiency. This behavior is due to strong **data dependencies between diagonals**, frequent synchronization, and barrier overhead, which limit the amount of exploitable parallelism. The row-wise parallel version shows **slight speedup up to 2–4 threads**, but performance degrades beyond this point due to dependency chains between rows and increased overhead.

VTUNE STATISTICS:-

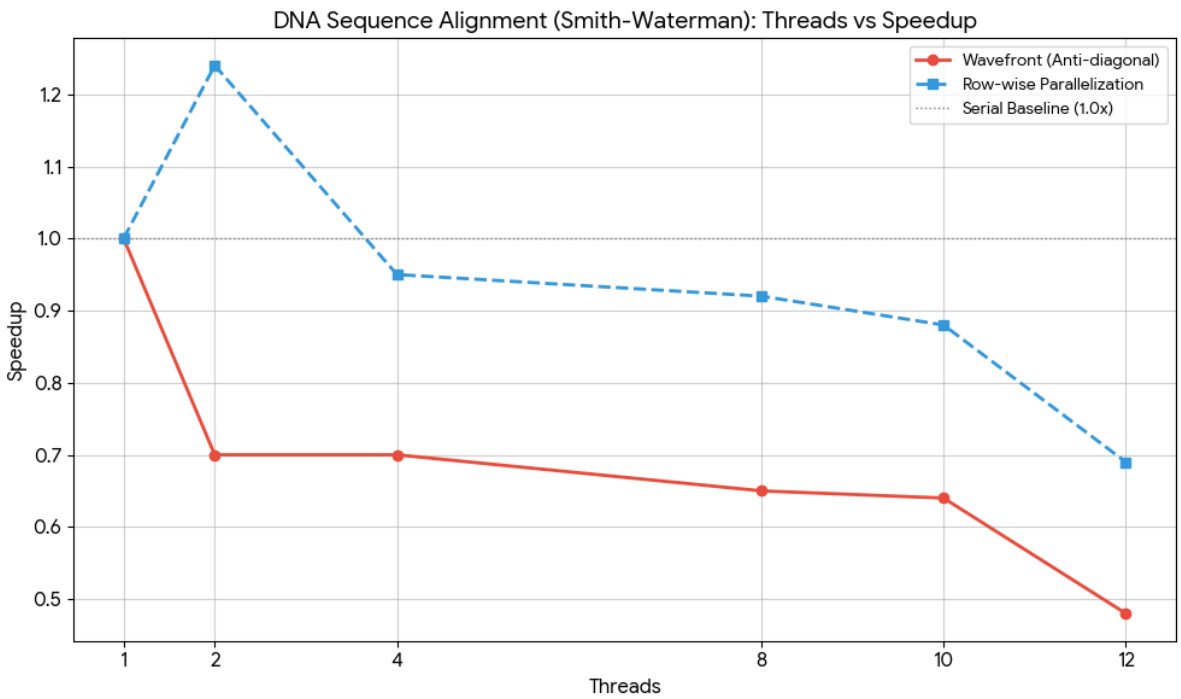
Metric	Observed Value	Interpretation
CPI (Cycles Per Instruction)	~0.90 – 0.97	Moderate CPI; instructions execute efficiently but stalls occur due to dependencies
Effective Physical Core Utilization	~20–32% (≈2–2.5 out of 8 cores)	Low utilization due to limited parallelism and synchronization
Effective Logical Core Utilization	~25–30% (≈3–3.5 out of 12 threads)	Hyperthreading provides minimal benefit
Memory Bound	~30–40% of pipeline slots	Significant waiting due to memory and cache accesses
Cache Bound	~30–50% of clock cycles	Major portion of time spent fetching data from cache
L1 Cache Bound	~10–20%	Frequent access to L1 cache
L2 Cache Bound	~3–5%	Minor contribution to stalls
L3 Cache Bound	~20–30%	Cache contention and working set size limit scalability
DRAM Bound	~0%	Application is not limited by main memory (RAM)
Observed DRAM Bandwidth	~3–6 GB/s	Well below platform peak (~56–58 GB/s)
LLC (Last Level Cache) Miss Count	~8.7×10^5	Indicates noticeable cache misses in DP matrix access
Average Memory Latency	~10–12 cycles	Confirms cache-level memory access
Vectorization	0%	No vectorization due to branches and dependencies
GFLOPS	~0	Algorithm is memory- and control-flow bound

VTune *HPC Performance* analysis reports a CPI close to **0.9**, indicating reasonable instruction execution efficiency. However, **effective physical core utilization remains low (around 20–30%)**,

confirming that available CPU cores are underutilized. This is caused by limited parallel regions and frequent waiting among threads.

Memory analysis shows that the application is **memory and cache bound**, with a significant fraction of pipeline slots stalled on cache accesses, especially L1 and L3 cache. DRAM bandwidth usage remains far below the platform peak, indicating that the program is not limited by main memory bandwidth but by cache locality and access patterns. The presence of cache misses and dependency-driven access patterns further restrict scalability. Additionally, no vectorization is observed, as the algorithm consists mainly of scalar operations with conditional branches and data dependencies.

Overall, this experiment demonstrates that although OpenMP can be applied to the Smith–Waterman algorithm, **algorithmic dependencies significantly limit parallel scalability**. HPC techniques provide limited performance benefits for this problem, and performance is primarily constrained by memory access behavior, synchronization overhead, and inherent data dependencies rather than raw computational power.



Q.3>Scientific Computing - Heat Diffusion Simulation:-

EXECUTION TABLE:-

```
piyush@piyush-LOQ-15IAX9:~/Documents/UCS645/LAB2$ ./q3
2D Heat Diffusion Simulation
Grid size: 512 x 512
Time steps: 100
Stability criterion (α*Δt/Δx²): 0.001 (must be < 0.25)

=== static Scheduling ===
Threads    Time (s)        Speedup         Efficiency      Avg Temp
-----------------------------------------------------------------------
1          0.558566        1.00            x100.0           %3.11°C
2          0.292494        1.91            x95.5            %3.11°C
4          0.166396        3.36            x83.9            %3.11°C
8          0.148844        3.75            x46.9            %3.11°C
10         0.118737        4.70            x47.0            %3.11°C
12         0.110202        5.07            x42.2            %3.11°C

=== dynamic Scheduling ===
Threads    Time (s)        Speedup         Efficiency      Avg Temp
-----------------------------------------------------------------------
1          0.567293        1.00            x100.0           %3.11°C
2          0.357177        1.59            x79.4            %3.11°C
4          0.198569        2.86            x71.4            %3.11°C
8          0.134580        4.22            x52.7            %3.11°C
10         0.122173        4.64            x46.4            %3.11°C
12         0.115916        4.89            x40.8            %3.11°C

=== guided Scheduling ===
Threads    Time (s)        Speedup         Efficiency      Avg Temp
-----------------------------------------------------------------------
1          0.537493        1.00            x100.0           %3.11°C
2          0.298723        1.80            x90.0            %3.11°C
4          0.152325        3.53            x88.2            %3.11°C
8          0.122931        4.37            x54.7            %3.11°C
10         0.098599        5.45            x54.5            %3.11°C
12         0.089964        5.97            x49.8            %3.11°C

=== Cache-Blocked Version (Block size: 32) ===
Threads    Time (s)        Speedup         Efficiency
-------------------------------------------------------
1          0.492411        1.00            x100.0%
2          0.259413        1.90            x94.9%
4          0.141874        3.47            x86.8%
8          0.122782        4.01            x50.1%
10         0.098954        4.98            x49.8%
12         0.116623        4.22            x35.2%
```

```
Performance counter stats for './q3':

   56,854,820,834      cpu_atom/cycles/                                                      (26.92%)
   77,944,368,904      cpu_core/cycles/                                                      (73.08%)
  165,242,897,524      cpu_atom/instructions/          #    2.91  insn per cycle             (26.92%)
  279,521,616,777      cpu_core/instructions/          #    3.59  insn per cycle             (73.08%)
      229,041,302      cpu_atom/cache-references/                                            (26.92%)
      427,413,754      cpu_core/cache-references/                                            (73.08%)
        2,931,847      cpu_atom/cache-misses/          #    1.28% of all cache refs          (26.92%)
        6,618,487      cpu_core/cache-misses/          #    1.55% of all cache refs          (73.08%)
   15,518,444,790      cpu_atom/branches/                                                    (26.92%)
   26,262,912,681      cpu_core/branches/                                                    (73.08%)

      5.390246580 seconds time elapsed

     18.409684000 seconds user
      0.140133000 seconds sys
```

PERFORMANCE STATISTICS:- (perf stat)

The 2D heat diffusion simulation shows **good parallel scalability** due to its regular grid structure and absence of data dependencies between grid points. Each thread updates independent cells, which eliminates race conditions and allows effective OpenMP parallelization. Among scheduling strategies, **guided scheduling performs the best**, achieving a speedup of ~6× with 12 threads, while static scheduling shows lower overhead for uniform workloads. The cache-blocked version further improves execution time by enhancing data locality.

VTUNE STATISTCS:-

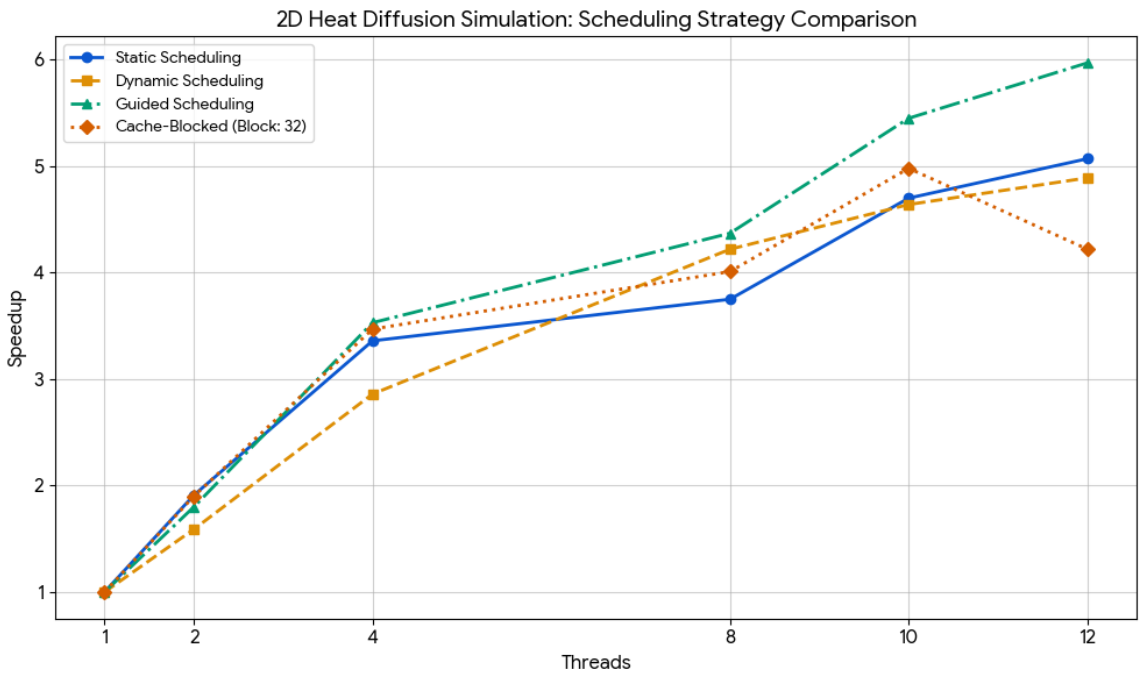
Metric	Observed Value	Interpretation
Speedup (12 threads)	~6.0×	Good parallel scalability
Efficiency (12 threads)	~50%	Reasonable overhead at higher threads
CPI	0.039 – 0.142	Very efficient instruction execution
Effective Physical Core Utilization	~18–23%	Parallelism limited by stencil structure
Memory Bound	~7–10%	Low memory stalls
Cache Bound	Dominant within memory stalls	Cache-level access dominates
DRAM Bound	< 1%	Not limited by main memory
Observed DRAM Bandwidth	~7–9 GB/s	Far below peak → memory not saturated
LLC Miss Count	~0	Excellent cache locality
Average Memory Latency	~3 cycles	Fast cache hits
Vectorization	~0–12%	Limited SIMD usage
GFLOPS (DP)	~0.4	Moderate floating-point intensity

VTune analysis indicates that the application is **compute-dominant rather than memory-bound**. Memory Bound remains low (~7–10%), while DRAM stalls are negligible, confirming that most data accesses are served from cache. Blocking improves temporal locality by reusing neighboring grid values inside cache, which explains the reduced execution time in the blocked version.

The **CPI values (≈0.04–0.14)** are very low, indicating efficient instruction execution with minimal pipeline stalls. Retiring dominates pipeline slots (>80%), meaning the CPU spends most cycles doing useful work. Vectorization is reported as low, suggesting that the compiler did not aggressively use SIMD instructions due to loop structure and boundary conditions.

Overall, q3 demonstrates a **well-parallelized stencil computation** where performance benefits come mainly from **thread-level parallelism, cache blocking, and proper scheduling**, rather than raw

memory bandwidth or vectorization. This makes the heat diffusion problem highly suitable for shared-memory parallel architectures.



Key Observations:

- **Best Overall Performer:** The **Guided Scheduling** strategy (green triangle line) consistently achieves the highest speedup, peaking at **5.97×** with 12 threads.
- **Static vs. Dynamic:** Static scheduling performs slightly better than dynamic scheduling at lower thread counts, but dynamic catches up at higher counts.
- **Cache-Blocked Decline:** Interestingly, the cache-blocked version (red diamond line) starts strong but sees a performance dip at 12 threads (**4.22×**), likely due to the overhead of managing blocks becoming less efficient as thread count increases or cache thrashing.